

Real-world (compare your loop to the library's) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below reuse `get_weather` and `celsius_to_fahrenheit` from `build_react_loop`, and assume `run_agent()` from that exercise's Solution is already in scope.

Basic Version

```
# agent_executor_comparison_practice.py
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

@tool
def get_weather_tool(city: str) -> str:
    """Get the current weather for a city, in Celsius."""
    return get_weather(city)

llm = ChatOpenAI(model="gpt-4o-mini")
agent = create_agent(llm, tools=[get_weather_tool])

result = agent.invoke({"messages": [("user", "What's the weather in Paris?")]}))
print(result["messages"][-1].content)
```

Expected output (final line; use `.stream(inputs, stream_mode="updates")` instead of `.invoke()` if you want to watch each step happen live):

```
It's 18°C and cloudy in Paris right now.
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — 3 test questions, both loops, diffed

```
# agent_executor_comparison_practice.py
test_questions = [
    "What's the weather in Paris?",
    "What's the weather in Paris, in Fahrenheit?",
    "What's the capital of France?",
]

for question in test_questions:
    mine = run_agent(question)
    lib = agent.invoke({"messages": [("user", question)]}, {"recursion_limit": 11})

    tool_call_count = sum(1 for m in lib["messages"] if getattr(m, "name", None))

    print("QUESTION:", question)
    print("  mine:", mine.final_answer, "|", len(mine.steps), "tool call(s)")
    print("  lib: ", lib["messages"][-1].content, "|", tool_call_count, "tool call(s)")
    print()
```

Expected output (shape; exact wording varies run to run):

```
QUESTION: What's the weather in Paris?
  mine: It's 18°C and cloudy in Paris right now. | 1 tool call(s)
  lib:  It's 18°C and cloudy in Paris right now. | 1 tool call(s)

QUESTION: What's the weather in Paris, in Fahrenheit?
  mine: It's 64.4°F in Paris. | 2 tool call(s)
  lib:  It's 64.4°F in Paris right now. | 2 tool call(s)

QUESTION: What's the capital of France?
  mine: The capital of France is Paris. | 0 tool call(s)
  lib:  The capital of France is Paris. | 0 tool call(s)
```

Approach 2 — counting the library's steps directly, not just eyeballing the stream

```
# agent_executor_comparison_practice.py
result = agent.invoke(
    {"messages": [("user", "What's the weather in Paris, in Fahrenheit?")]},
    {"recursion_limit": 11},
)
print(result["messages"][-1].content)

tool_messages = [m for m in result["messages"] if getattr(m, "name", None)]
print("lib step count:", len(tool_messages))
```

`result["messages"]` holds the whole conversation, including every tool call and its result — filtering for messages that have a `name` set (a tool's own return message) gives you the library's own equivalent of your `steps` list, so you can compare step *counts* precisely instead of eyeballing a printed stream. **Expected output:**

```
It's 64.4°F in Paris right now.
lib step count: 2
```

Difference from Basic: Basic runs one question through the library loop alone. Approach 1 runs the *same* 3 questions through both loops and prints them side by side, so you can actually compare, not just observe each one in isolation. Approach 2 replaces "reading a printed stream" with reading `result["messages"]` directly — the more reliable way to compare step counts, since streamed output is meant for humans to read, not for code to parse.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a failing tool, compared side by side

```
# agent_executor_comparison_practice.py
call_count = {"n": 0}

@tool
def flaky_weather_tool(city: str) -> str:
    """Get the current weather for a city. Fails on the first call."""
    call_count["n"] += 1
    if call_count["n"] == 1:
        raise RuntimeError("weather service timed out")
    return get_weather(city)

flaky_agent = create_agent(llm, tools=[flaky_weather_tool])
result = flaky_agent.invoke({"messages": [("user", "What's the weather in Paris?")]}
print(result["messages"][-1].content)
```

Expected behavior: the agent built by `create_agent` catches the `RuntimeError` on its own and feeds the exception's text back to the model as a tool-result message — the model then typically retries, and the second call succeeds. Your own hand-built `run_tool()` from `build_react_loop`'s Advanced Version does the same *shape* of thing (catch, feed back as Observation, let the model react) — but it feeds back the exact error message you chose (`f"Error: {e}"`), which may not be word-for-word what the library sends. This is the concrete answer to Hint 1 Advanced's question: both recover, but the observation text the model sees can differ, and that difference can change what the model does next.

Approach 2 — hitting the step limit, compared side by side

```
# agent_executor_comparison_practice.py
try:
    result = agent.invoke(
        {"messages": [("user", "What's the weather in Paris, in Fahrenheit?")]}},
        {"recursion_limit": 2},
    )
    print("lib result at limit:", result["messages"][-1].content)
except Exception as e:
```

```

    print("lib raised:", type(e).__name__, e)

try:
    run_agent("What's the weather in Paris, in Fahrenheit?", max_iterations=1)
except MaxIterationsExceeded as e:
    print("mine raised:", e)

```

Expected output:

```

lib raised: GraphRecursionError Recursion limit of 2 reached without hitting a st
mine raised: No answer after 1 steps

```

The real difference: your `run_agent()` raises `MaxIterationsExceeded` — the Build Task requires this ("raised, not silently ignored"). A `create_agent`-built agent also raises, on its own, once `recursion_limit` is hit — `GraphRecursionError`, since it's built on `LangGraph` underneath — so both loops now agree on "fail loudly instead of quietly returning a stopped-early message as if it were a real answer." That agreement is itself worth noting: the older `AgentExecutor` used to default to returning quietly instead, which is exactly the kind of surprising difference this exercise exists to catch — the library's behavior has moved closer to your own hand-built loop's, not further away.

Difference from Intermediate: Intermediate compares final answers and step counts on the happy path, where both loops behave almost identically. Advanced compares the 2 places they diverge — a tool failure, and the step limit — and finds a real, practical point to check: the exact error text the model sees on a tool failure, since that's the piece your own loop still controls more precisely than the library does.

Which one should you actually use? For Project 2 and most real work: `create_agent` (or hand-built `LangGraph`, covered later) — it's tested, handles edge cases you'd otherwise reinvent, and having built `run_agent()` yourself means you now know what it's doing well enough to trust it, and precisely enough to know where to look when it does something unexpected. Keep your own hand-built loop only as a reference for exactly this kind of comparison, or for a case needing a specific behavior — like a very particular error message on a tool failure — that you want full control over instead of accepting whatever the library sends by default.